

Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 06

Norma (Módulo) de un Vector en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: May 12, 2026

Contents

1	Introducción	2
2	Organización del proyecto	3
3	Código fuente	5
3.1	Programa principal — <code>main.cpp</code>	5
3.2	Módulo ensamblador — <code>vectorNormal.asm</code>	5
4	Instrucciones x86 utilizadas	7
5	El rol de los registros	8
5.1	Marco de pila y parámetros	8
5.2	Registros del procesador	8
5.3	Estado de la pila FPU por instrucción	8
6	Ejemplo de ejecución	9
6.1	Segundo ejemplo: vector no entero	10
7	Repositorio GitHub	10
8	Conclusiones	10

1 Introducción

Esta práctica consiste en implementar el cálculo de la **norma euclídea** (también llamada módulo o longitud) de un vector de números reales, utilizando **lenguaje ensamblador x86** con la **Unidad de Punto Flotante (FPU)**. La función ensamblador es invocada desde un programa principal escrito en **C++**, que se encarga de solicitar los datos al usuario y presentar el resultado con seis decimales de precisión.

El objetivo va más allá de reproducir la operación aritmética: se busca comprender cómo la FPU acumula sumas de cuadrados en su pila interna, cómo se ejecuta una raíz cuadrada a nivel de hardware mediante la instrucción **FSQRT**, y cómo el procesador recorre un arreglo contiguo en memoria avanzando un puntero registro a registro.

¿Qué es la norma de un vector?

Dado un vector $\vec{v} = (v_1, v_2, \dots, v_N)$ de dimensión N , su **norma euclídea** (norma-2 o módulo) se define como:

$$\|\vec{v}\| = \sqrt{\sum_{i=1}^N v_i^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_N^2} \quad (1)$$

Geométricamente, la norma representa la **longitud del vector** en el espacio n -dimensional. Por ejemplo, para un vector de dimensión 3:

$$\|(3, 4, 0)\| = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16} = \sqrt{25} = 5 \quad (2)$$

La norma siempre produce un **escalar no negativo**, y es igual a cero únicamente para el vector nulo.

¿Cómo se representan los vectores en memoria?

En C++, un `vector<double>` almacena sus elementos de forma **contigua** en memoria. Cada elemento de tipo `double` ocupa **8 bytes** (64 bits, formato IEEE 754 de doble precisión). Si el primer elemento está en la dirección base D , los elementos sucesivos se encuentran en:

Elemento	Dirección	Desplazamiento
<code>vec[0]</code>	D	+0 bytes
<code>vec[1]</code>	$D + 8$	+8 bytes
<code>vec[2]</code>	$D + 16$	+16 bytes
<code>vec[i]</code>	$D + 8i$	+8 <i>i</i> bytes

Table 1: Distribución en memoria de un arreglo de `double`

El ensamblador recorre estos elementos avanzando el puntero **ESI** en 8 bytes por iteración con la instrucción **ADD ESI, 8**.

2 Organización del proyecto

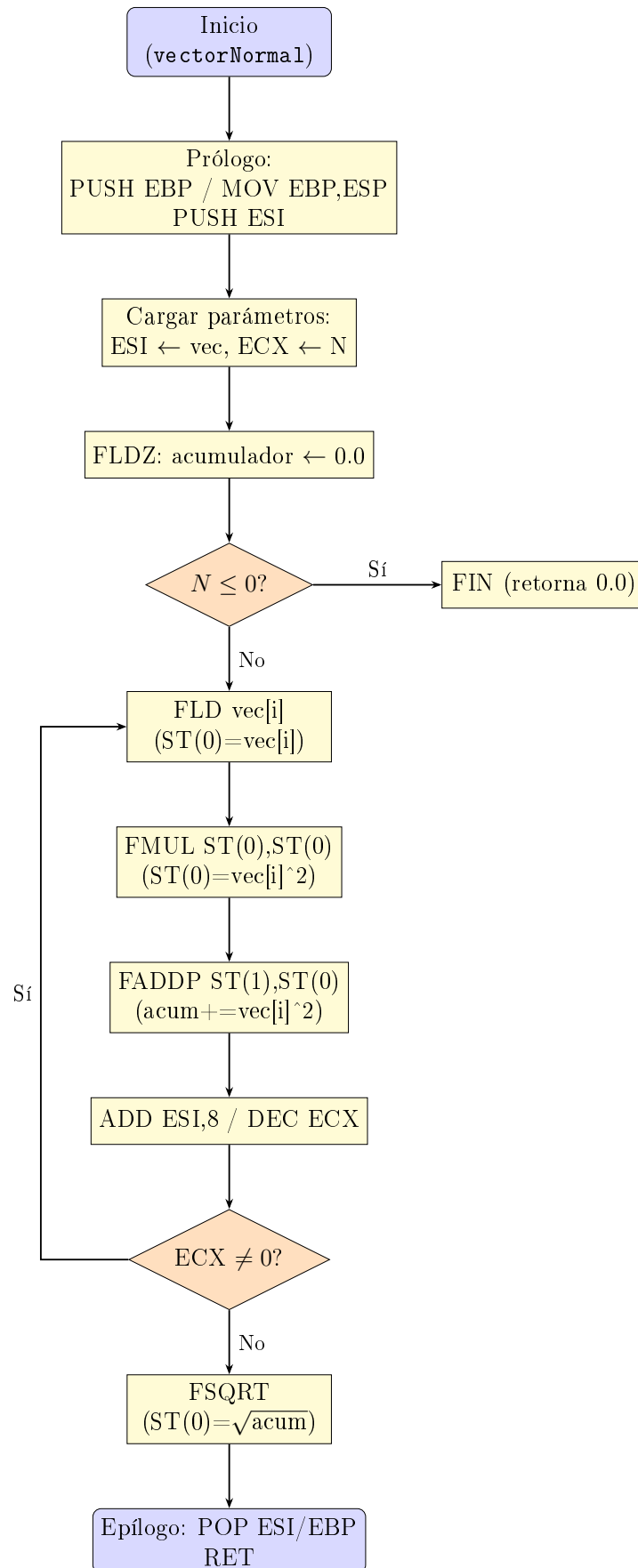
El proyecto está configurado como una aplicación de consola Win32 en Visual Studio 2022 (conjunto de herramientas v145) e integra dos módulos que se enlazan en tiempo de compilación:

Archivo	Función
<code>vectorNormal.asm</code>	Implementa en ensamblador x86 la función <code>vectorNormal</code> , que recibe un puntero al vector y su dimensión N , acumula la suma de cuadrados en la pila FPU y devuelve la raíz cuadrada como <code>double</code> en <code>ST(0)</code> .
<code>main.cpp</code>	Programa principal en C++ que solicita la dimensión y los elementos del vector al usuario, invoca la función ensamblador y muestra la norma con seis decimales de precisión.
<code>Practica06_VectorNormal.vcxproj</code>	Proyecto de Visual Studio que habilita MASM (<code>masm.props</code> / <code>masm.targets</code>) para ensamblar el archivo <code>.asm</code> como parte del proceso de construcción normal.

Table 2: Archivos del proyecto

La directiva `.model flat, c` y la declaración `extern "C"` en C++ garantizan la compatibilidad de nombres (sin *name-mangling*) entre ambos módulos. El resultado `double` se retorna en `ST(0)` de la FPU, conforme a la convención de llamada *cdecl* de x86.

El algoritmo se divide en dos etapas bien delimitadas:

Figure 1: Diagrama de flujo del algoritmo `vectorNormal`

3 Código fuente

3.1 Programa principal — main.cpp

```
1 // =====
2 // Autor: Edson Joel Carrera Avila
3 // main.cpp
4 // =====
5
6 #include <iostream>
7 #include <iomanip>
8 #include <vector>
9
10 using namespace std;
11
12 extern "C" double vectorNormal(double* vec, int N);
13
14 int main() {
15     int n;
16
17     cout << "Ingresa la dimension del vector (N): ";
18     cin >> n;
19
20     vector<double> vec(n);
21
22     cout << "\n--- DATOS DEL VECTOR ---" << endl;
23     for (int i = 0; i < n; i++) {
24         cout << "Ingresa el valor para vec[" << i << "]: ";
25         cin >> vec[i];
26     }
27
28     double norma = vectorNormal(vec.data(), n);
29
30     cout << "\n-----\n";
31     cout << "Resultado de la Norma del Vector: "
32         << fixed << setprecision(6) << norma << endl;
33
34     return 0;
35 }
```

Listing 1: main.cpp – Programa principal en C++

3.2 Módulo ensamblador — vectorNormal.asm

```
1 ; =====
2 ; Autor: Edson Joel Carrera Avila
3 ; vectorNormal.asm
4 ; =====
5
6 .586
7 .model flat, c
8
```

```

9 ; =====
10 ; SECCIÓN DE CÓDIGO (.code)
11 ; =====
12 .code
13 ; --- Inicio del programa ---
14
15 ; Parámetros: [EBP+8] = puntero a vec (double*)
16 ;               [EBP+12] = N (int)
17 vectorNormal PROC
18     PUSH     EBP                ; Guardamos el marco de pila actual
19     MOV      EBP, ESP          ; Establecemos el nuevo marco de
20         pila
21     PUSH     ESI                ; Preservamos ESI según la convención
22         n cdecl
23     MOV      ESI, [EBP+8]        ; ESI apunta al inicio de vec
24     MOV      ECX, [EBP+12]      ; ECX almacena la dimensión N (
25         contador del ciclo)
26     FLDZ                    ; Cargamos 0.0 en la pila FPU. ST(0)
27         = acumulador
28     TEST     ECX, ECX          ; Comparamos N con 0
29     JLE      fin              ; Si N <= 0, saltamos al final (
30         retorna 0.0)
31
32 ; --- Bucle de suma de cuadrados ---
33 bucle:
34     FLD      QWORD PTR [ESI]    ; Cargamos vec[i] en ST(0). El
35         acumulador baja a ST(1)
36     FMUL     ST(0), ST(0)       ; Multiplicamos ST(0) por sí mismo.
37         ST(0) = vec[i]^2
38     FADDP    ST(1), ST(0)       ; Sumamos ST(0) al acumulador en ST
39         (1) y desapilamos
40     ADD      ESI, 8             ; Avanzamos 8 bytes en vec (tamaño
41         de un double)
42     DEC      ECX               ; Disminuimos el contador N
43     JNZ      bucle             ; Si ECX no es 0, repetimos el ciclo
44
45 ; --- Raíz cuadrada ---
46     FSQRT                    ; ST(0) = sqrt(suma_cuadrados)
47
48 ; --- Fin del programa ---
49 fin:
50     POP      ESI
51     MOV      ESP, EBP
52     POP      EBP
53     RET
54 vectorNormal ENDP
55
56 END

```

Listing 2: vectorNormal.asm – Función ensamblador x86

4 Instrucciones x86 utilizadas

Instrucción	Qué hace
PUSH / POP	Coloca o retira un valor de 32 bits de la pila del sistema; se usan en el prólogo/epílogo y para preservar <code>ESI</code> según la convención <i>cdecl</i> .
MOV	Copia un valor entre registros o entre registro y memoria.
ADD	Suma un valor inmediato a un registro; aquí avanza el puntero <code>ESI</code> en 8 bytes por iteración para acceder al siguiente <code>double</code> .
DEC	Decrementa el operando en 1; actualiza las banderas del procesador, lo que permite a <code>JNZ</code> detectar el fin del bucle.
TEST	Realiza una AND lógica sin guardar el resultado; solo actualiza las banderas para verificar si <code>ECX</code> es cero antes del primer ciclo.
JLE	Salta si el resultado indica “menor o igual” (<i>jump if less or equal</i>); protege el bucle cuando $N \leq 0$.
JNZ	Salta si el resultado no es cero (<i>jump if not zero</i>); mantiene el bucle mientras el contador <code>ECX</code> $\neq 0$.
RET	Retorna al llamador (C++); el resultado <code>double</code> permanece en <code>ST(0)</code> de la FPU.
FLDZ	Carga la constante 0.0 al tope de la pila FPU (<code>ST(0)</code>); inicializa el acumulador de la suma de cuadrados.
FLD	Carga (<i>push</i>) un valor <code>QWORD</code> (64 bits / <code>double</code>) desde memoria al tope de la pila FPU; desplaza <code>ST(0)</code> a <code>ST(1)</code> .
FMUL	Multiplica <code>ST(0)</code> por el operando indicado; en este caso <code>ST(0), ST(0)</code> eleva al cuadrado el valor recién cargado.
FADDP	Suma <code>ST(0)</code> a <code>ST(1)</code> y hace <i>pop</i> ; el acumulador queda actualizado en lo que era <code>ST(1)</code> , que pasa a ser el nuevo <code>ST(0)</code> .
FSQRT	Calcula la raíz cuadrada del valor en <code>ST(0)</code> de forma nativa en hardware y almacena el resultado en el mismo registro; no requiere iteraciones por software.

Table 3: Instrucciones x86 y FPU utilizadas en `vectorNormal`

5 El rol de los registros

5.1 Marco de pila y parámetros

Al momento en que C++ invoca `vectorNormal`, coloca los dos argumentos en la pila del sistema en orden inverso (de derecha a izquierda, convención *cdecl*). Después del prólogo `PUSH EBP / MOV EBP, ESP`, el mapa de la pila queda:

Dirección	Contenido	Tipo
[EBP+12]	N	int (4 bytes)
[EBP+8]	vec	double* (4 bytes en x86)
[EBP+4]	dirección de retorno	—
[EBP+0]	EBP anterior	—

Table 4: Mapa del marco de pila al inicio de `vectorNormal`

A diferencia de la práctica anterior (producto punto con dos vectores), aquí solo se recibe **un puntero**, lo que simplifica el prólogo: únicamente es necesario preservar ESI, ya que EDI no se utiliza.

5.2 Registros del procesador

Registro	Rol	Descripción
ESI	Puntero al vector	Apunta al elemento actual de <code>vec</code> ; avanza 8 bytes por iteración para recorrer el arreglo contiguo en memoria.
ECX	Contador de ciclo	Almacena N e inicia la cuenta regresiva hasta 0; al llegar a cero, <code>JNZ</code> no salta y el bucle termina.
ST(0)	Tope de la pila FPU	Valor temporal: <code>vec[i]</code> recién cargado o su cuadrado; al terminar el bucle contiene la suma de cuadrados y después de <code>FSQRT</code> contiene la norma.
ST(1)	Segundo registro FPU	Acumulador: suma parcial de v_i^2 ; solo existe mientras hay un elemento cargado en ST(0).

Table 5: Registros del procesador utilizados en `vectorNormal`

5.3 Estado de la pila FPU por instrucción

La pila FPU tiene profundidad máxima de 8 registros (ST(0)–ST(7)). En este algoritmo se utiliza una profundidad máxima de 2. La siguiente tabla muestra cómo evoluciona en las instrucciones clave del bucle:

Instrucción	ST(0)	ST(1)	Descripción
FLDZ	0.0	vacío	Inicializa el acumulador.
FLD [ESI]	v_i	acum	Carga el elemento; el acumulador baja a ST(1).
FMUL ST(0),ST(0)	v_i^2	acum	Eleva al cuadrado sin mover la pila.
FADDP ST(1),ST(0)	$\text{acum} + v_i^2$	vacío	Suma y hace <i>pop</i> ; la pila vuelve a profundidad 1.
FSQRT	$\sqrt{\text{acum}}$	vacío	Raíz cuadrada en hardware; profundidad 1.

Table 6: Estado de la pila FPU durante la ejecución de `vectorNormal`

6 Ejemplo de ejecución

Para el vector $\vec{v} = (3, 4, 0)$, el cálculo esperado es:

$$\|\vec{v}\| = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{9 + 16 + 0} = \sqrt{25} = 5.000000 \quad (3)$$

La tabla siguiente muestra la evolución del acumulador iteración a iteración:

Iteración	vec[i]	vec[i] ²	Acumulador (ST)	ECX
0 (inicial)	—	—	0.0	3
1	3.0	9.0	$0 + 9 = 9.0$	2
2	4.0	16.0	$9 + 16 = 25.0$	1
3	0.0	0.0	$25 + 0 = 25.0$	0
FSQRT	—	—	$\sqrt{25} = 5.000000$	—

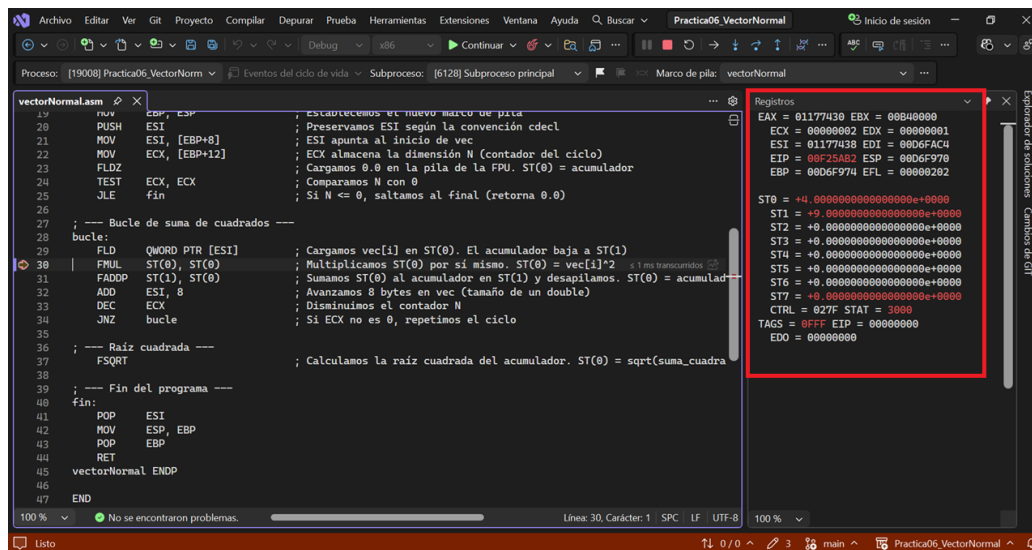
Table 7: Evolución del acumulador FPU para $\vec{v} = (3, 4, 0)$ 

Figure 2: Ventana *Registers* de Visual Studio: registros FPU ST(0)–ST(7) y ECX durante el bucle de acumulación de cuadrados.

```

Consola de depuración de Mi
Ingresa la dimension del vector (N): 3
--- DATOS DEL VECTOR ---
Ingresa el valor para vec[0]: 3
Ingresa el valor para vec[1]: 4
Ingresa el valor para vec[2]: 0
-----
Resultado de la Norma del Vector: 5.000000
C:\Users\ING EJCA\Desktop\Practica06_VectorNormal\Debug\Practica06_VectorNormal.exe (proceso 19008) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la con
sola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .]

```

Figure 3: Salida en consola del programa para $\vec{v} = (3, 4, 0)$: norma resultante igual a 5.000000.

6.1 Segundo ejemplo: vector no entero

Para reforzar la precisión de punto flotante, se verifica con $\vec{v} = (1, 1, 1)$:

$$\|(1, 1, 1)\| = \sqrt{1 + 1 + 1} = \sqrt{3} \approx 1.732051 \quad (4)$$

```

Consola de depuración de Mi
Ingresa la dimension del vector (N): 3
--- DATOS DEL VECTOR ---
Ingresa el valor para vec[0]: 1
Ingresa el valor para vec[1]: 1
Ingresa el valor para vec[2]: 1
-----
Resultado de la Norma del Vector: 1.732051
C:\Users\ING EJCA\Desktop\Practica06_VectorNormal\Debug\Practica06_VectorNormal.exe (proceso 12332) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la con
sola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .]

```

Figure 4: Salida en consola para $\vec{v} = (1, 1, 1)$: la FPU calcula correctamente la norma irracional $\sqrt{3} \approx 1.732051$.

7 Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica06_VectorNormal

8 Conclusiones

- La norma de un vector ilustra cómo el procesador puede ejecutar algoritmos matemáticos completos —incluyendo potenciación, acumulación y raíz cuadrada— **íntegramente mediante la FPU**, sin depender de bibliotecas de punto flotante de software. La instrucción FSQRT delega la operación directamente a hardware dedicado, lo que la hace eficiente y precisa.
- El patrón FLD $\rightarrow$ FMUL ST(0), ST(0) $\rightarrow$ FADDP ST(1), ST(0) mantiene la pila FPU con profundidad máxima de 2 durante todo el bucle: un elemento para el valor al cuadrado y otro para el acumulador. Esta gestión cuidadosa evita desbordamientos

de la pila de 8 niveles y resulta más eficiente que cargar y descargar el acumulador en cada iteración.

- La diferencia respecto al **producto punto** (Práctica 05) pone de manifiesto la flexibilidad de la FPU: donde antes se necesitaban dos punteros (ESI y EDI) y una multiplicación cruzada entre vectores, aquí basta con **un solo puntero** y una auto-multiplicación (FMUL ST(0),ST(0)), demostrando que pequeñas variaciones en el algoritmo tienen un impacto directo en el número de registros utilizados.
- La **verificación de $N \leq 0$** con TEST y JLE es indispensable: si se omitiera y se pasara un valor negativo, DEC / JNZ ejecutarían el bucle aproximadamente 2^{32} veces, leyendo memoria fuera del arreglo y produciendo resultados indefinidos o corrupción de datos.
- Preservar únicamente ESI (y no EDI, que no se usa) muestra el principio de **mínima perturbación**: la convención *cdecl* obliga a guardar los registros que se modifiquen, pero no los que permanecen intactos, reduciendo el número de operaciones de pila innecesarias.